ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

старший преподаватель		Д.О. Шевяков
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №8

МОДУЛЬ АНАЛИЗА ДАННЫХ В СИСТЕМАХ ИНТЕРНЕТА ВЕЩЕЙ

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4321		И.А. Лаврешин
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков анализа данных в системах интернета вещей.

2 Задание

Создать систему анализа данных, хранящихся в базе. Система должна рассчитывать, как минимум две различных характеристики, например среднее и максимальное значение.

3 Ход выполнения задания

В класс Logger добавлены методы average_temperature и max_temperature: обход документов коллекции Temperature (или списка в памяти), извлечение поля Temperature, вычисление среднего арифметического и максимума. Добавлен маршрут GET /stats, возвращающий JSON с avg_temperature и max_temperature. В ответе GET /connect к полям ответа добавлены те же агрегаты для отображения после каждой установки температуры. Результат работы представлен на рисунке 1.

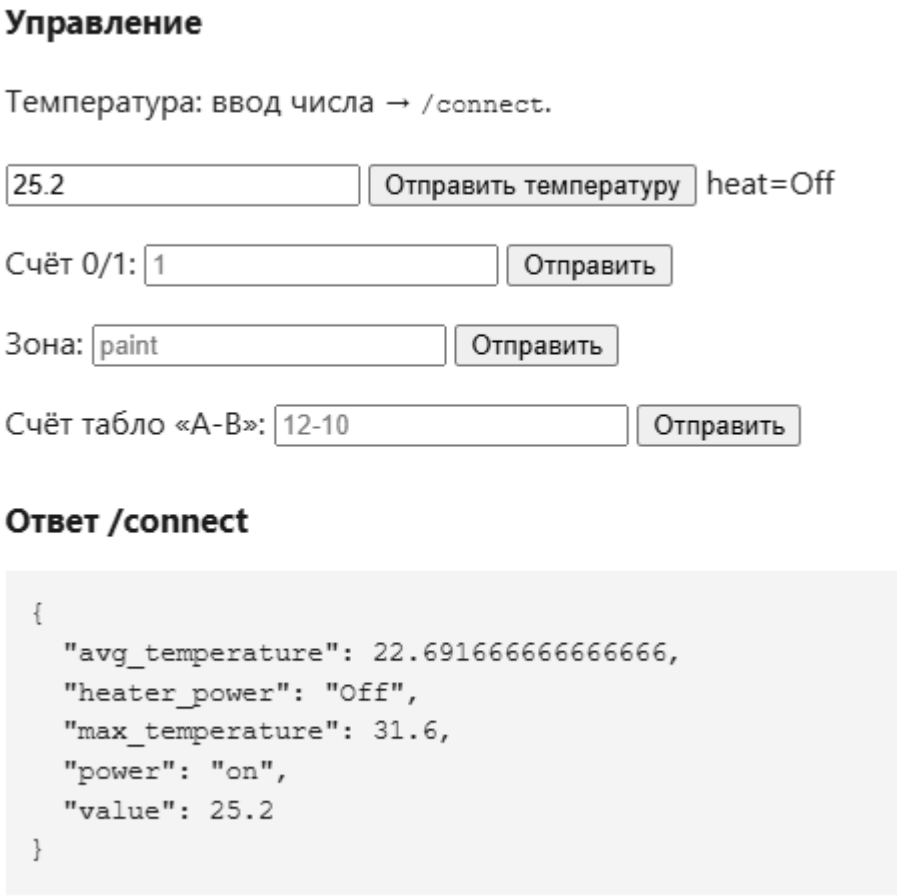

Рисунок 1 – Результат работы программы

4 Вывод

Реализованы две числовые характеристики ряда измерений температуры – среднее и максимум, доступные через API и в составе ответа /connect.

ПРИЛОЖЕНИЕ А

Листинг с кодом программы

```
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="utf-8" />
    <title>Эмуляторы вещей</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/main.css') }}" />
    <script src="https://code.jquery.com/jquery-3.3.1.min.js"></script>
    <script src="{{ url_for('static', filename='js/index.js') }}"></script>
  </head>
  <body>
    <p><a href="{{ url_for('index') }}">← На главную</a></p>
    <h1>Мониторинг и команды</h1>
    <section>
      <h2>Мониторинг</h2>
      <div class="row">
        <label>Температура °C <input type="text" id="mon_env" readonly
/></label>
        <label>Счёт (гол) <input type="text" id="mon_score" readonly /></label>
      </div>
      <div class="row">
        <label>Зона <input type="text" id="mon_zone" readonly /></label>
        <label>Табло <input type="text" id="mon_board" readonly /></label>
      </div>
    </section>

    <section>
      <h2>Управление</h2>
      <p>Температура: ввод числа → <code>/connect</code>.</p>
```

```
      <input type="text" id="cmd_env" placeholder="23.5" />
      <button type="button" id="btn_env">Отправить температуру</button>
      <span id="cmd_env_reply"></span>

      <p>Счёт 0/1: <input type="text" id="cmd_score" placeholder="1" /> <button type="button" id="btn_score">Отправить</button></p>
      <p>Зона: <input type="text" id="cmd_zone" placeholder="paint" /> <button type="button" id="btn_zone">Отправить</button></p>
      <p>Счёт табло «A-B»: <input type="text" id="cmd_board" placeholder="12-10" /> <button type="button" id="btn_board">Отправить</button></p>
    </section>

    <section>
      <h2>Ответ /connect</h2>
      <pre id="chain_out"></pre>
    </section>
  </body>
</html>

//
from __future__ import annotations
from flask import Flask, jsonify, render_template, request
import things
app = Flask(__name__)
env_sensor = things.EnvSensor('es1', 'Микроклимат зала', 22.0, 400.0)
score_sensor = things.ScoreSensor('ss1', 'Кольцо команды A', is_goal=False)
zone_sensor = things.ZoneSensor('zs1', 'Разметка площадки', 'three-point-arc')
scoreboard = things.Scoreboard('sb1', 'Табло арены', 0, 0, 0)
heater = things.Heater('Обогреватель зала', 22.0)
logger = things.Logger('iot_basketball_db')
```

```
store = things.CourtDataStore()
mcu = things.MainControlUnit([score_sensor, zone_sensor, env_sensor, scoreboard], store, scoreboard)

def demo_iot_cycle() -> None:
    print('=== collectData ===')
    mcu.collectData()
    print('=== calculateScore ===')
    mcu.calculateScore()
    print('=== updateScoreboard ===')
    mcu.updateScoreboard()
    print('=== getRecords ===', len(store.getRecords()))

@app.route('/monitor/env')
def monitor_env():
    return jsonify(env_sensor.connect())

@app.route('/monitor/score')
def monitor_score():
    return jsonify(score_sensor.connect())

@app.route('/monitor/zone')
def monitor_zone():
    return jsonify(zone_sensor.connect())

@app.route('/monitor/board')
def monitor_board():
    return jsonify(scoreboard.connect())

@app.route('/connect')
```

```
def connect():
    response = env_sensor.connect_with_commands(request)
    heater.auto_power(env_sensor.temperature)
    logger.insert_temperature(env_sensor.temperature)
    logger.insert_heater_event(heater.power)
    response['heater_power'] = heater.power
    avg = logger.average_temperature()
    mx = logger.max_temperature()
    response['avg_temperature'] = avg
    response['max_temperature'] = mx
    return jsonify(response)

@app.route('/connect_heater')
def connect_heater():
    return jsonify(heater.connect())

@app.route('/command/score')
def command_score():
    return jsonify(score_sensor.connect_with_commands(request))

@app.route('/command/zone')
def command_zone():
    return jsonify(zone_sensor.connect_with_commands(request))

@app.route('/command/board')
def command_board():
    return jsonify(scoreboard.connect_with_commands(request))

@app.route('/stats')
def stats():
```

```
    return jsonify({'avg_temperature': logger.average_temperature(), 'max_temperature': logger.max_temperature()})

@app.route('/lab2')
def lab2_page():
    demo_iot_cycle()
    return render_template('lab2.html')

@app.route('/')
def index():
    return render_template('index.html')

@app.route('/emulator')
def emulator_page():
    return render_template('sensor_emulator.html')
if __name__ == '__main__':
    app.run(debug=True)

//
from __future__ import annotations
import abc
import datetime
import random
import re
from typing import Any, Dict, List, Optional
try:
    import pymongo
except ImportError:
    pymongo = None
```

```
class Thing(abc.ABC):

    def __init__(self, thing_id: str, name: str) -> None:
        self.id = thing_id
        self.name = name

    @abc.abstractmethod
    def getStatus(self) -> str:
        pass

    def emulate(self) -> None:
        pass

    def connect(self, request: Any=None) -> Dict[str, Any]:
        raise NotImplementedError

class ScoreSensor(Thing):

    def __init__(self, thing_id: str, name: str, is_goal: bool=False) -> None:
        super().__init__(thing_id, name)
        self.is_goal = is_goal

    def getStatus(self) -> str:
        return 'ScoreSensor[' + self.id + ']: is_goal=' + str(self.is_goal)

    def emulate(self) -> None:
        self.is_goal = random.choice([True, False])

    def connect(self, request: Any=None) -> Dict[str, Any]:
        self.emulate()
```

```
        return {'id': self.id, 'value': 1 if self.is_goal else 0, 'is_goal': self.is_goal}

    def connect_with_commands(self, request: Any) -> Dict[str, Any]:
        if request is not None:
            raw = request.args.get('value', '')
            try:
                v = int(float(raw))
                self.is_goal = bool(v)
            except ValueError:
                print('ScoreSensor: ожидалось число 0/1, получено:',
type(raw).__name__)
        return {'power': 'on', 'is_goal': self.is_goal}

class ZoneSensor(Thing):
    ZONES = ('three-point-arc', 'paint', 'midcourt', 'baseline')

    def __init__(self, thing_id: str, name: str, zone: str) -> None:
        super().__init__(thing_id, name)
        self.zone = zone

    def getStatus(self) -> str:
        return 'ZoneSensor[' + self.id + ']: zone=' + self.zone

    def emulate(self) -> None:
        self.zone = random.choice(self.ZONES)

    def connect(self, request: Any=None) -> Dict[str, Any]:
        self.emulate()
        return {'id': self.id, 'value': self.zone, 'zone': self.zone}
```

```
    def connect_with_commands(self, request: Any) -> Dict[str, Any]:
        if request is None:
            return {'zone': self.zone}
        raw = request.args.get('value', '') or ''
        if re.fullmatch('[a-z0-9\\-]+', raw):
            self.zone = raw
        else:
            print('ZoneSensor: недопустимая зона (нужны латиница, цифры, дефис):', repr(raw))
        return {'power': 'on', 'zone': self.zone}

class EnvSensor(Thing):

    def __init__(self, thing_id: str, name: str, temperature: float, light_lux: float) -> None:
        super().__init__(thing_id, name)
        self.temperature = float(temperature)
        self.lightLux = float(light_lux)

    def getStatus(self) -> str:
        return 'EnvSensor[' + self.id + ']: t=' + str(round(self.temperature, 1)) + ', lux=' + str(int(self.lightLux))

    def emulate(self) -> None:
        self.temperature = round(random.uniform(18.0, 28.0), 1)
        self.lightLux = float(random.randint(300, 600))

    def connect(self, request: Any=None) -> Dict[str, Any]:
        self.emulate()
```

```
        return {'id': self.id, 'value': self.temperature, 'temperature': self.temperature,
'lightLux': int(self.lightLux)}

    def connect_with_commands(self, request: Any) -> Dict[str, Any]:
        if request is None:
            return {'power': 'on', 'value': self.temperature}
        raw = request.args.get('value', '')
        try:
            self.temperature = float(raw)
            self.temperature = round(self.temperature, 1)
        except ValueError:
            print('EnvSensor: не удалось привести к float, тип:', type(raw).__name__,
'значение:', repr(raw))
        return {'power': 'on', 'value': self.temperature}

class Scoreboard(Thing):

    def __init__(self, thing_id: str, name: str, score_a: int=0, score_b: int=0,
timer_sec: int=0) -> None:
        super().__init__(thing_id, name)
        self.scoreA = score_a
        self.scoreB = score_b
        self.timerSec = timer_sec

    def getStatus(self) -> str:
        return 'Scoreboard[' + self.id + ']: ' + str(self.scoreA) + ':' + str(self.scoreB) + ',
t=' + str(self.timerSec)

    def emulate(self) -> None:
        self.timerSec += random.randint(0, 2)
```

```
        if random.random() < 0.3:
            self.scoreA += 1

    def connect(self, request: Any=None) -> Dict[str, Any]:
        self.emulate()
        return {'id': self.id, 'value': self.scoreA, 'scoreA': self.scoreA, 'scoreB':
self.scoreB, 'timer': self.timerSec}

    def connect_with_commands(self, request: Any) -> Dict[str, Any]:
        if request is None:
            return {'scoreA': self.scoreA, 'scoreB': self.scoreB}
        raw = request.args.get('value', '')
        try:
            parts = raw.split('-')
            if len(parts) == 2:
                self.scoreA = int(parts[0])
                self.scoreB = int(parts[1])
        except ValueError:
            print('Scoreboard: ожидался формат A-B для счёта')
        return {'power': 'on', 'scoreA': self.scoreA, 'scoreB': self.scoreB}

class Heater:

    def __init__(self, name: str, switch_on_temperature: float) -> None:
        self.name = name
        self.power = 'Off'
        self.switch_on_temperature = switch_on_temperature

    def connect(self) -> Dict[str, Any]:
        return {'heater_power': self.power}
```

```
    def auto_power(self, temperature: float) -> None:
        self.power = 'On' if temperature < self.switch_on_temperature else 'Off'

class Logger:

    def __init__(self, db_name: str) -> None:
        self.db_name = db_name
        self._mongo_db: Any = None
        self._last_temp: Optional[float] = None
        self._client: Any = None
        self._mem_temp: List[Dict[str, Any]] = []
        self._mem_events: List[Dict[str, Any]] = []
        if pymongo is None:
            print('Logger: pymongo не установлен, используется память.')
            return
        try:
            self._client = pymongo.MongoClient('mongodb://127.0.0.1:27017/',
serverSelectionTimeoutMS=2500)
            self._client.admin.command('ping')
            self._mongo_db = self._client[db_name]
            print('Logger: подключение к MongoDB OK, база', db_name)
        except Exception as exc:
            print('Logger: MongoDB недоступна (', exc, '), используется память.')

    def insert_temperature(self, new_data: float) -> Optional[Any]:
        if new_data == self._last_temp:
            print('Logger: температура не изменилась, запись пропущена')
            return None
        self._last_temp = float(new_data)
```

```
        ts = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        doc = {'timeStamp': ts, 'Temperature': float(new_data)}
        if self._mongo_db is not None:
            return self._mongo_db['Temperature'].insert_one(doc)
        self._mem_temp.append(doc)
        return doc

    def insert_heater_event(self, power_state: str) -> Optional[Any]:
        ts = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        doc = {'timeStamp': ts, 'heater_power': power_state}
        if self._mongo_db is not None:
            return self._mongo_db['HeaterEvents'].insert_one(doc)
        self._mem_events.append(doc)
        return doc

    def _iter_temperature_docs(self) -> List[Dict[str, Any]]:
        if self._mongo_db is not None:
            return list(self._mongo_db['Temperature'].find())
        return list(self._mem_temp)

    def average_temperature(self) -> Optional[float]:
        docs = self._iter_temperature_docs()
        vals = [d.get('Temperature') for d in docs if d.get('Temperature') is not None]
        if not vals:
            return None
        return sum(vals) / len(vals)

    def max_temperature(self) -> Optional[float]:
        docs = self._iter_temperature_docs()
        vals = [d.get('Temperature') for d in docs if d.get('Temperature') is not None]
```

```
        if not vals:
            return None
        return max(vals)

class CourtDataStore:

    def __init__(self) -> None:
        self.events: List[str] = []
        self.telemetry: List[str] = []

    def saveEvent(self, data: Any) -> None:
        self.events.append('event=' + repr(data))

    def saveTelemetry(self, data: Any) -> None:
        self.telemetry.append('telemetry=' + repr(data))

    def getRecords(self) -> List[str]:
        return self.events + self.telemetry

class MainControlUnit:

    def __init__(self, things: List[Thing], store: CourtDataStore, scoreboard: Scoreboard) -> None:
        self.things = things
        self.store = store
        self.scoreboard = scoreboard

    def collectData(self) -> None:
        for t in self.things:
            self.store.saveTelemetry({'id': t.id, 'status': t.getStatus()})
```

```
    def calculateScore(self) -> None:
        for t in self.things:
            if isinstance(t, ScoreSensor) and t.is_goal:
                self.scoreboard.scoreA += 1
                self.store.saveEvent({'type': 'basket', 'points': self.scoreboard.scoreA})
                break

    def updateScoreboard(self) -> None:
        self.scoreboard.timerSec += 1
function poll(url, id, pick) {
  $.ajax({
    type: "GET",
    url: url,
    dataType: "json",
    contentType: "application/json",
    data: {},
    success: function (response) {
      document.getElementById(id).value = pick(response);
    },
  });
}

function startMonitoring() {
  setInterval(function () {
    poll("/monitor/env", "mon_env", function (r) {
      return r.temperature != null ? r.temperature : r.value;
    });
  }, 1000);
```

```
  setInterval(function () {
    poll("/monitor/score", "mon_score", function (r) {
      return r.is_goal ? "гол" : "нет";
    });
  }, 1000);

  setInterval(function () {
    poll("/monitor/zone", "mon_zone", function (r) {
      return r.zone;
    });
  }, 1000);

  setInterval(function () {
    poll("/monitor/board", "mon_board", function (r) {
      return r.scoreA + ":" + r.scoreB + " t=" + r.timer;
    });
  }, 1000);
}

function sendCommand(url, valueFieldId, extra) {
  var v = document.getElementById(valueFieldId).value;
  $.ajax({
    type: "GET",
    url: url,
    dataType: "json",
    data: { value: v },
    success: function (response) {
      if (extra) extra(response);
    },
  });
```

```
}

$(function () {
  startMonitoring();

  $("#btn_env").on("click", function () {
    sendCommand("/connect", "cmd_env", function (r) {
      var h = r.heater_power;
      $("#cmd_env_reply").text(
        h !== undefined && h !== null && String(h) !== "" ? "heat=" + h : ""
      );
      $("#chain_out").text(JSON.stringify(r, null, 2));
    });
  });

  $("#btn_score").on("click", function () {
    sendCommand("/command/score", "cmd_score");
  });
  $("#btn_zone").on("click", function () {
    sendCommand("/command/zone", "cmd_zone");
  });
  $("#btn_board").on("click", function () {
    sendCommand("/command/board", "cmd_board");
  });
});
```